Objectives

1. Develop a simple and easy-to-use User Interface (UI) for the clinical data warehouse.
2. Use a version control system to store and maintain the program.
3. Follow good coding practices to ensure that the code is readable and easy to maintain.

Description

You are working as an Information Technology staff for a local hospital. As the hospital continues to grow, the clinical data warehouse you developed is used by more users, who do not necessarily have the technical expertise to directly start and interact with it through command lines on a terminal. Therefore, you are asked to develop a simple and easy-to-use UI, through which users can interact with your program to add, remove, retrieve patient information, or perform other tasks. In addition, as the program grows, you are also asked to properly store and maintain the program code using a version control system so that your team members can access and modify the code for continued development and integration.

Developing a simple UI

You should use the TKinter package (https://docs.python.org/3/library/tkinter.html) to implement the UI.

Essential functionalities through the UI

1. **Validating credentials:** when the program starts and the UI is shown to users, it should show a log-in page and prompt users to enter their username and password and click log in.
2. **Reading in existing patient information:** if a user's username and password match the records in the credential file, the UI should read in a file that contains existing patient information (input file 2) and show action buttons. The menu shown to a user should depend on the type of the user. For example, if the user that logged in is in the management type, then the only two buttons shown should be "Generate key statistics" and "Exit". If the user that logged in is in the nurse or clinician type, then six buttons should be displayed to them: "Retrieve_patient", "Add_patient", "Remove_patient", "Count_visits", "View_Note", and "Exit".
3. **Retrieving, adding, and removing patient information:** After the user clicks on a button, the UI should prompt the user to enter information corresponding to the action required. For example, if the user clicks on the "Retrieve_patient" button, the UI should then prompt the user to enter a Patient_ID and display the patient information in an easily readable format (you can assume that this information should be from the patient's most recent visit if there are multiple visits). After a user performs the action and is shown results, the user should be returned to the menu where they can select one of the actions again. If a user clicks on "Remove_patient", then the user should be prompted to enter a patient ID.
4. **View note:** If a user logged in as a nurse or clinician, they should also be able to click on the view_note button and then be prompted to enter a patient ID and date to select a note to read from the patient on that date. The UI should then display the note selected by the user in a readable format.
5. **Generating key statistics and count visits:** If a user logged in is in the management or admin role, then the only action the user can perform is to generate key statistics, count visits, monitor provider workload and department revenue. After the user clicks on this button, your program

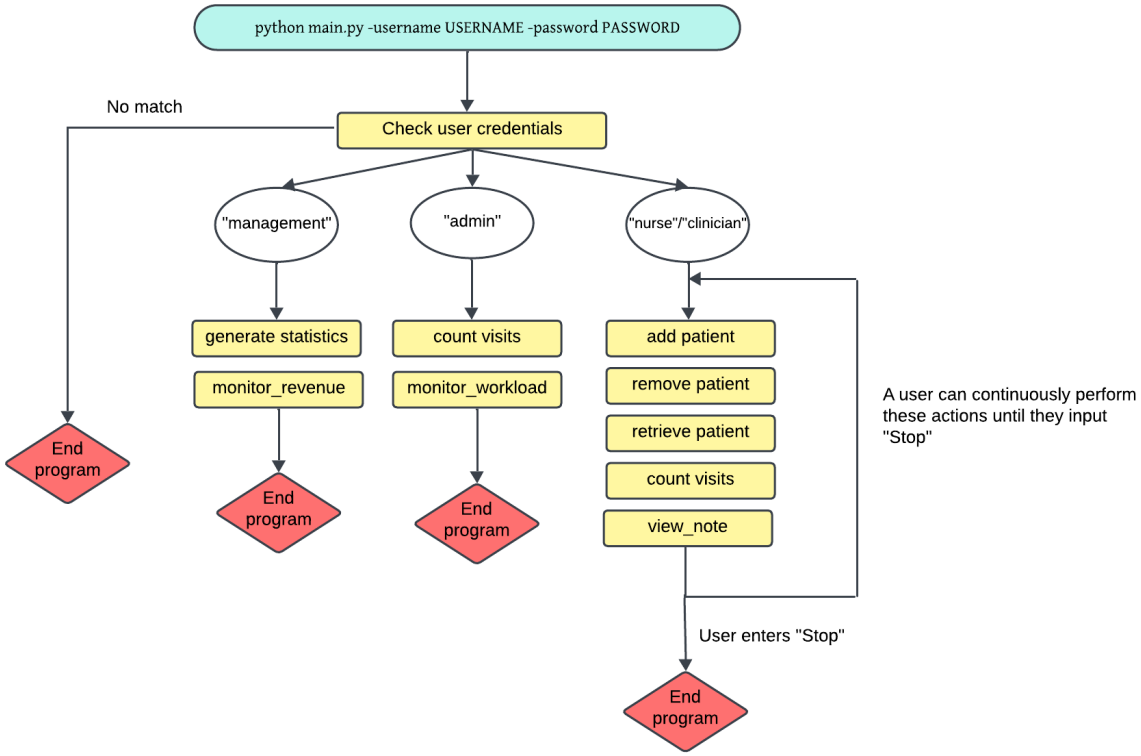
should either display key statistics on the screen through the UI, or write and generate key statistics to a file and alert the user about the file.

6. **Exiting the program** from UI by clicking on a "Exit" button. This should terminate the program.

Table 1: Four user roles

Value	Description
"admin"	Administrative staff. **They cannot access PHI**. They can access non-PHI but not patient-related information. Therefore, their possible actions are: count_encounters_per_patient, count_encounters_by_department, monitor_provider_workload.
"clinician"	Healthcare professionals. They can access all patient information and perform all actions.
"nurse"	Healthcare professionals. They can access all patient information and perform all actions.
"management"	Human resource managers who do not interact with patients. **They cannot access PHI**. Therefore, their possible action is: monitor_department_revenue.

Once a user can access patient information, your program should allow interactions including add_patient, remove_patient, retrieve_patient, view_note, count_encounters_per patient, count_encounters_by_department, identify eligible patients. Note that the possible data and interactions for the user should be strictly based on their roles, as described above. For example, the program should never prompt a user with "admin" or "management" role to access PHI.



Generating usage statistics

Now that as more users are using the software, the IT department plans to track usage of the software so that they have more insight into which types of users are frequently using the software and for what types of tasks. After each user finishes using the software, your program should store the information such as the username and role for the user, the action the user performed, and time of log-in. This information should be stored in a file and updated after a user has used the program. The file should be in a format that is easily readable and accessible to management staff who are not familiar with formats such as json.

Writing patient information to file

After users perform actions, such as removing or adding patient information, your program should store these changes to existing patient information and update the patient information file accordingly.

Patient Data

The complete patient data description is provided in Programming Assignment 2 handout.

Program implementation requirements

Your program for this assignment should be based on your code for Programming Assignment 1 & 2. You should also add another class to model the UI application.

Input files

For simplicity, your program should assume that the following input files are under a folder named **Data** which is located in the same directory as your Python program. You should not hard code the absolute path of these files. Instead, you can use the relative file path in your program (e.g., "./credentials.csv").

Input file 1: credential file

The file **credentials.csv** will contain usernames, passwords, and user roles of authorized users. Your program should read in this file first so that it can check users' credentials. The possible values included in the file are described on Page 2.

Input files 2: existing patient files

These are the files you used from Programming Assignment 2. Your program should also read it in so that it has all existing patient information.

Input file 3: clinical note file

This file contains the clinical notes for the patients. You can link the patients with their notes by using Note_ID.

The possible input from users with proper credentials include:

1. `Add_patient`: this adds a new patient to the program. After the user clicks on the button, your program should provide a menu for the user to input patient information and a button to submit the information. The menu should have an appropriate design for inputting different types of information, e.g., selection boxes for categorical information (e.g., gender) and blanks for free-text information (e.g., chief complaints). Your program should then prompt the user to enter Patient_ID. If the Patient_ID already exists, you should prompt the user to enter the Visit_time and randomly create a unique visit_ID and add the information to the patient records. If the Patient_ID does not exist, your program should store the new Patient_ID and prompt the user to enter other information. After the user provides information, your program should append the new patient information to the existing patient file.
2. `Remove_patient`: this removes all information about the patient from your program. Your program should then prompt the user enter Patient_ID. If the Patient_ID does not exist, your program should print an alert to the user to show this. Otherwise, remove all records associated with the Patient_ID. Your program should also delete the corresponding patient information from the patient file.
3. `Retrieve_patient`: your program should then prompt the user to a Patient_ID. Your program should search if this Patient_ID exists. If not, print an alert. If yes, the UI should then prompt the user to enter a Patient_ID and display the patient information in an easily readable format (you can assume that this information should be from the patient's most recent visit if there are multiple visits).
4. `Count_visits`: this function allows users to check how many patients were seen at a given date. Your program should prompt the user to enter a date following the format "YYYY-MM-DD" and then count the number of total visits (if a patient has multiple visits, count them all) on that day and display to the patient. **Note that users may choose to count visits per patient and by department, similar to in PA2.**

5. `View_note`: this function allows users to read a clinical note for a patient. Since a patient may have multiple visits and therefore multiple clinical notes, your program should prompt the user to enter a date (YYYY-MM-DD) and display notes that were for that date.
6. `Generate_key_statistics`: this should generate and display meaningful statistics from patient data. The plots should be easy to read and informative. You should decide what statistics will be informative to the audience.
7. `Monitor_revenue:` compute the total procedure costs generated by each department.
8. `Monitor_workload:` compute the number of encounters handled by clinicians and rank them by the number of encounters.

Grading (600 points in total)

The submission will be graded based on the following criteria. The assignment will be 400 points in total.

Program (200 points)

40 points: I can start your program through terminal using the command you provided in a README file and interact with the program through a UI.

120 points: For successfully implementing the 8 essential actions.

40 points: For successfully implementing the log in function and verifying user credentials and their roles.

Output files (100 points)

100 points: There are two expected output files. The first one is the patient information file. The final file should be updated to reflect the patient information after changes made by the users such as adding new patients and removing patients. The final file should follow the format of the original patient information file I provided on Canvas. The second output file is the usage statistics file. This file should record each log in made by users. This file should record the user's username, role, log in time, and actions performed. If a user fails to login by providing incorrect user credentials, this file should also record this event and mark it as failed log in attempt.

Coding style (50 points)

50 points: Your program should be easy to read. You should follow the general coding styles introduced in class, such as good naming practices, small and concise functions, etc.

Design (150 points)

40 points: The UI for your program should be simple, easy to use, and intuitive. I should be able to interact with the UI to perform the tasks easily.

40 points: You should submit a UML diagram that illustrates the design of your classes. The granularity of the diagram is up to you, but at least it should clearly show the classes, relationships among the classes, and attributes and methods of each class. The UML diagram should match the implementation of your program. **Your UML diagram should be produced from a diagramming tool (draw.io, Lucid Chart, Google Draw) and not from hand drawing or word documents.**

20 points: Your program should be structured into multiple .py files and a main file (main.py) that imports modules from other files. Putting all codes into one file will lose points. The structure of these files should also be carefully thought out, e.g., patients.py should contain patient-related functions.

However, the main file is the one that I will execute to start the program. All other files will not be directly run but should be imported into the main file to be used.

50 points: Your program should implement **at least 4 classes using object-oriented programming**, including the User class and a class for modeling the UI application. Each class has a set of attributes and methods and an appropriate constructor.

Documentation (50 points)

50 points: you should write a README file that explains: (1) how to run your file, such as the command used, and packages required. For packages required to run your program, you should use anaconda or other Python environment management tools to provide a requirement.txt or requirement.yaml so that I can directly replicate your environment. (2) a short description of your program, for other programmers who are not familiar with it; (3) other information that you think is important for users and programmers to be aware of. There are many good resources for how to write a ReadMe file online, e.g, https://dev.to/merlos/how-to-write-a-good-readme-bog.

Version control (50 points)

20 points: You should use GitHub to store and maintain the code base. Your submission should include a link to your GitHub repository that contains the code for this assignment. You need to make the repository a public repository so that I can view and download it. **You should show the code, not just the zip file on GitHub.**

20 points: Your GitHub repository structure should be clear and easy to understand. The repository should show your ReadMe on the front page and store the UML diagram and source code in the repository. You should also have a directory for storing the two output files. For best practices for structuring your GitHub repository, you can refer to: https://medium.com/code-factory-berlin/github-repository-structure-best-practices-248e6effc405, though for this project, you would need only a data folder, a src folder, a UML diagram, and a ReadME.

10 points: Your GitHub repository should show **at least one commit** that you made to make changes of your code. Simply uploading your code to a GitHub repository does not count. **You must make a commit with a commit message to make a modification to your code.** Any changes are acceptable. This is intended to test if you can use the basic functionality of GitHub for version control.

Submission

Your submission should be a public GitHub repository link in the submission comment so that I can access. Make sure that your repository is accessible to me or other users so that I can grade your assignment.

The assignment is due: 5/22/2026, 11:59pm CT. No extension allowed due to the grade submission deadline set by UWM.

Late submission

If you submit your assignment after the deadline but within 24 hours, only 70% of the overall points (140 points) will be considered for full. After 24 hours, all submissions will receive 0 points.

If you have personal or medical conditions that may affect submitting the assignment on time, please reach out to me as soon as possible to communicate an alternative timeline for submission.

Plagiarism

Discussion with your classmates is encouraged, but you should not directly duplicate their codes. **If direct duplication of codes is found, you will receive no points in this assignment.** You can use AI such as ChatGPT for troubleshooting or consulting, but you cannot directly copy and paste code generated from these tools. If you refer to code from external sources such as StackOverflow, please add the link to the comment of your program.